Build Sand Table Map

1. Course Content

Course Overview

Before a robot can autonomously navigate and perform long-process embodied tasks such as transport and sorting in a sandbox map, it must first build a set of high-precision **digital twin sandbox map and pose constraint graph** for the robot.

This system uses the industry-leading **Google Cartographer SLAM algorithm**:

1. **2D Grid Map (`map.yaml` / `map.pgm`)**: Presents the sandbox physical outline in black and white grid (black for walls/columns/shelves as obstacles, white for open drivable ground), providing a foundation for path planning and obstacle avoidance;
2. **Cartographer Pose Constraint Graph (`map.pbstream`)**: Records all non-linear loop topology constraints between the cart's laser scan points and submaps during mapping. The localization algorithm uses this `.pbstream` file to achieve second-level high-precision global repositioning, completely eliminating long-distance cumulative drift;
3. **Route Network Planning Topology Description (Road Network)**: Based on Cartographer localization, through calibrating waypoints and connections, planning the cart's high-speed cruise "main road" to prevent the cart from taking shortcuts or getting stuck in shelf gaps.

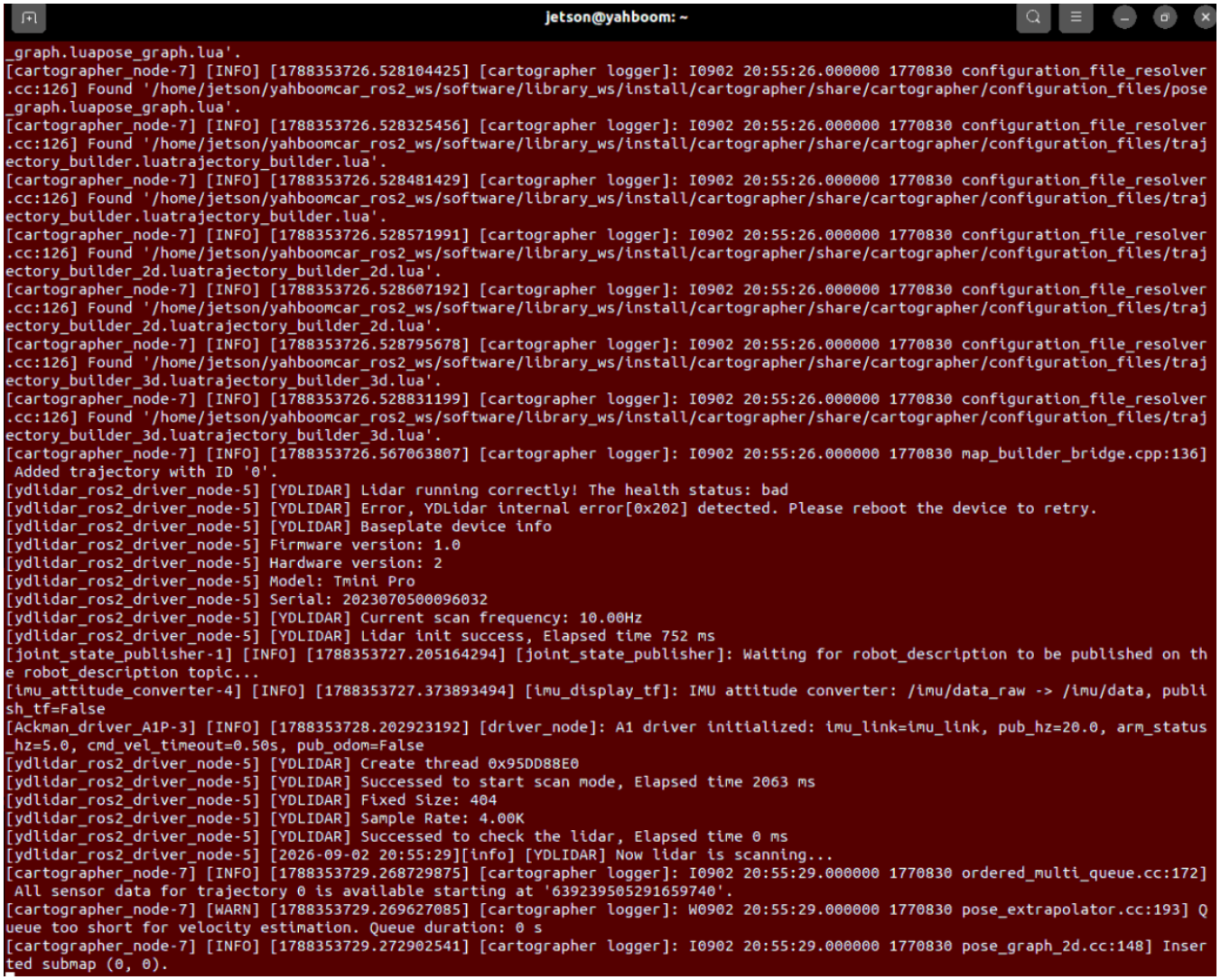
Learning Objectives

1. Master the complete process of building sandbox high-precision grid map and pose graph (`.pbstream`) using Cartographer algorithm.
2. Learn commands for saving 2D maps and calling `/write_state` service to save Cartographer pose graphs.
3. Learn to load `map.pbstream` for localization startup and use `Publish Point` tool to calibrate route network files.

2. Build Grid Map and Pose Map

Terminal 1: Start Cart Chassis and Cartographer Mapping Node (Cart Side)

```
ros2 launch yahboomcar_nav map_cartographer_launch.py
```



Terminal 2: Start RViz Mapping Visualization (RDK board and Raspberry Pi board recommend using virtual machine side)

```
ros2 launch yahboomcar_nav display_map_launch.py
```

Terminal 3: Start Keyboard Control Node

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

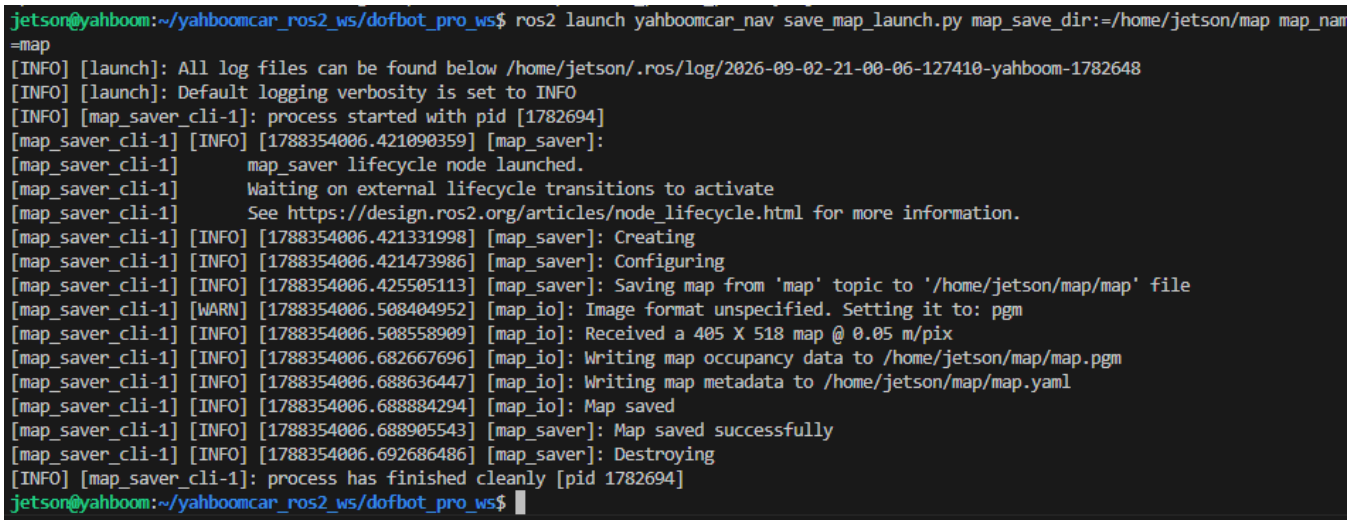
2.3 Save Grid Map and Pose Map

After mapping is complete, execute save commands in terminal in sequence:

1. Save 2D Grid Map (`map.yaml` + `map.pgm`)

```
# Save map to ~/map
ros2 launch yahboomcar_nav save_map_launch.py map_save_dir:=/home/jetson/map map_name:=map
#rdk and Raspberry Pi commands
ros2 launch yahboomcar_nav save_map_launch.py map_save_dir:=/root/map map_name:=map
ros2 launch yahboomcar_nav save_map_launch.py map_save_dir:=/home/sunrise/map map_name:=map
```

Terminal printing `Map saved successfully` indicates the 2D map was saved successfully:



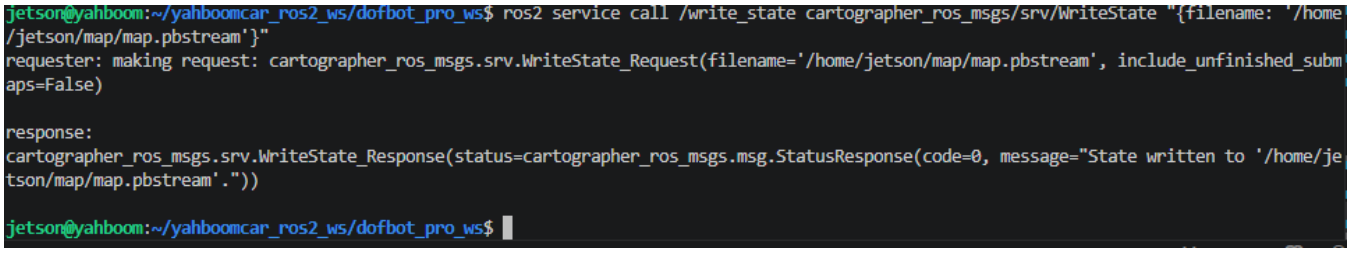
2. Save Cartographer Pose Graph (`map.pbstream`)

Through ROS 2 service call, write the current trajectory graph and submap constraints to `.pbstream` file:

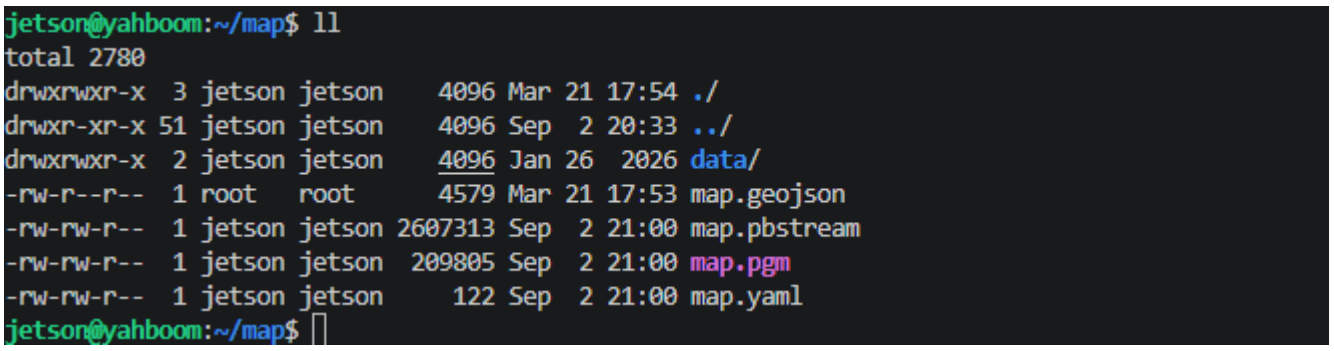
```
# RDK X5 system (sunrise user)
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename:
'/home/sunrise/map/map.pbstream'}"

# For host jetson user
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename:
'/home/jetson/map/map.pbstream'}"

# For host pi5 user
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename:
'/root/map/map.pbstream'}"
```



Map files are saved in `~/map` or specified directory by default, containing three core assets: `map.yaml`, `map.pgm`, and `map.pbstream`:



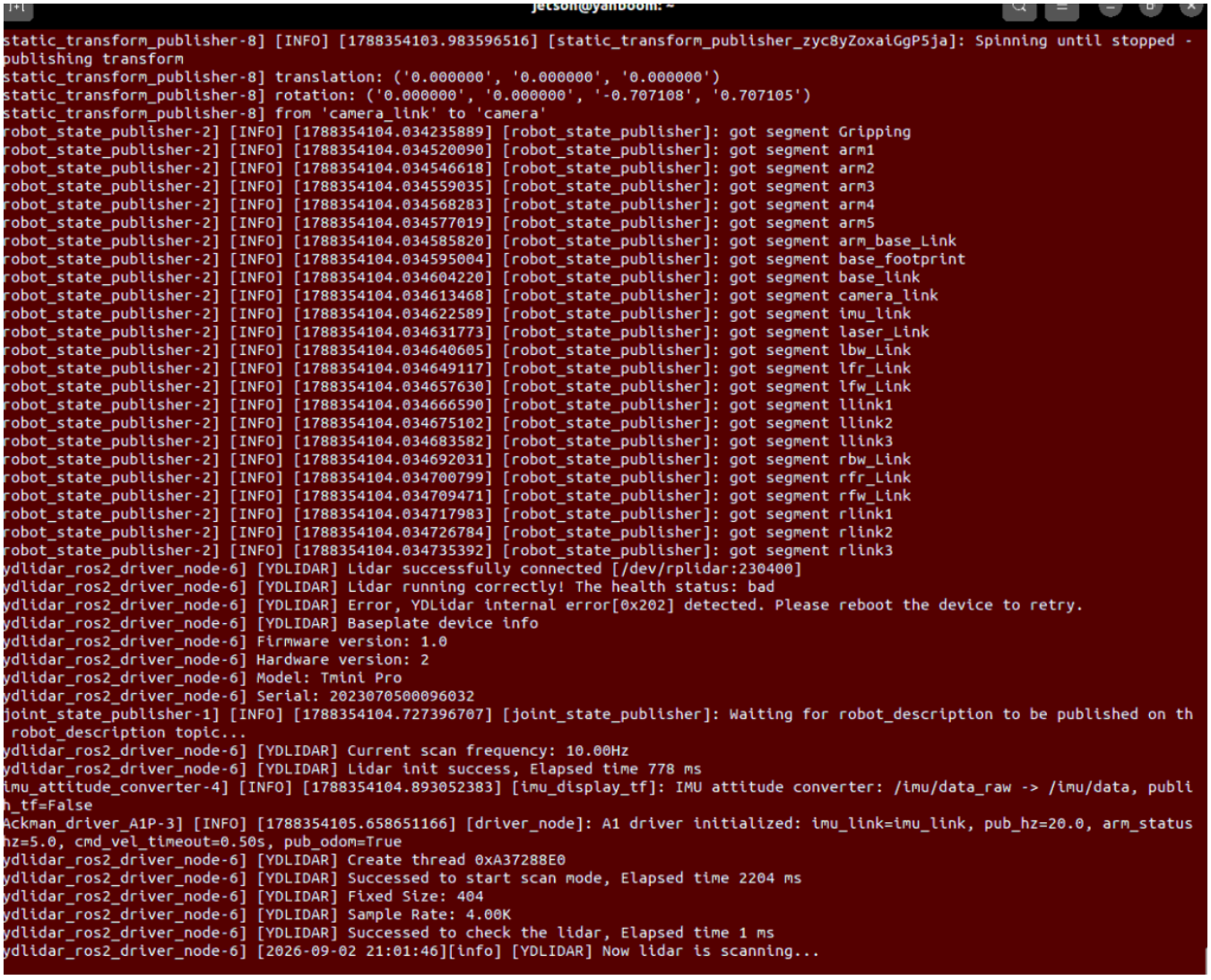
3. Phase 2: Route Annotation

3.1 Start Global Localization and Annotation Tools

When annotating route network, the system needs to know the cart's precise coordinates in the global map in real-time, so start chassis drive and navigation localization node first.

Terminal 1: Start Cart Chassis and LiDAR Driver (Cart Side)

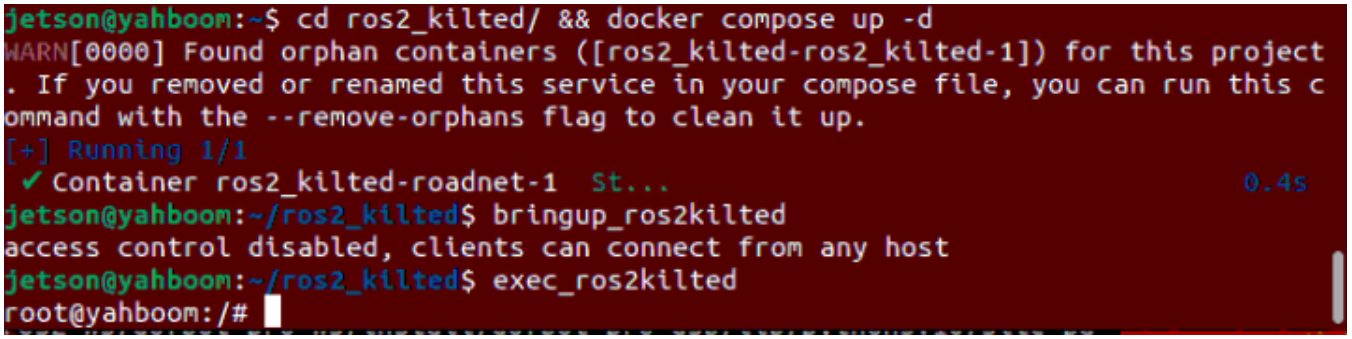
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



Terminal 2: Start Docker Container Environment (Optional/If Using Pre-installed Image)

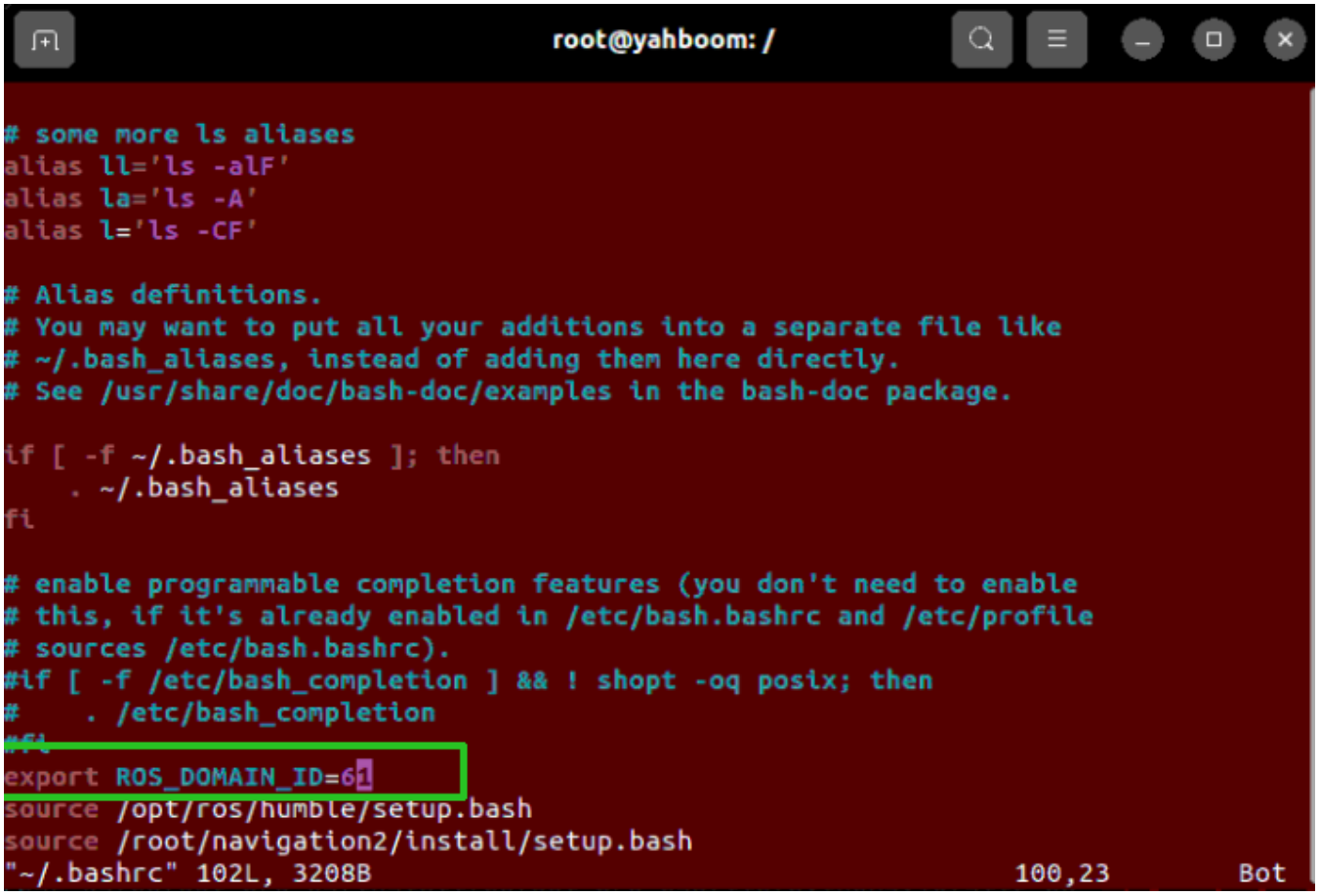
If using `ros2_kilted` container environment:

```
# Quick start command
bringup_ros2kilted
# Enter container terminal
exec_ros2kilted
```



In terminal input, ensure `ROS_DOMAIN_ID` is consistent between container and host:

```
vi ~/.bashrc
```



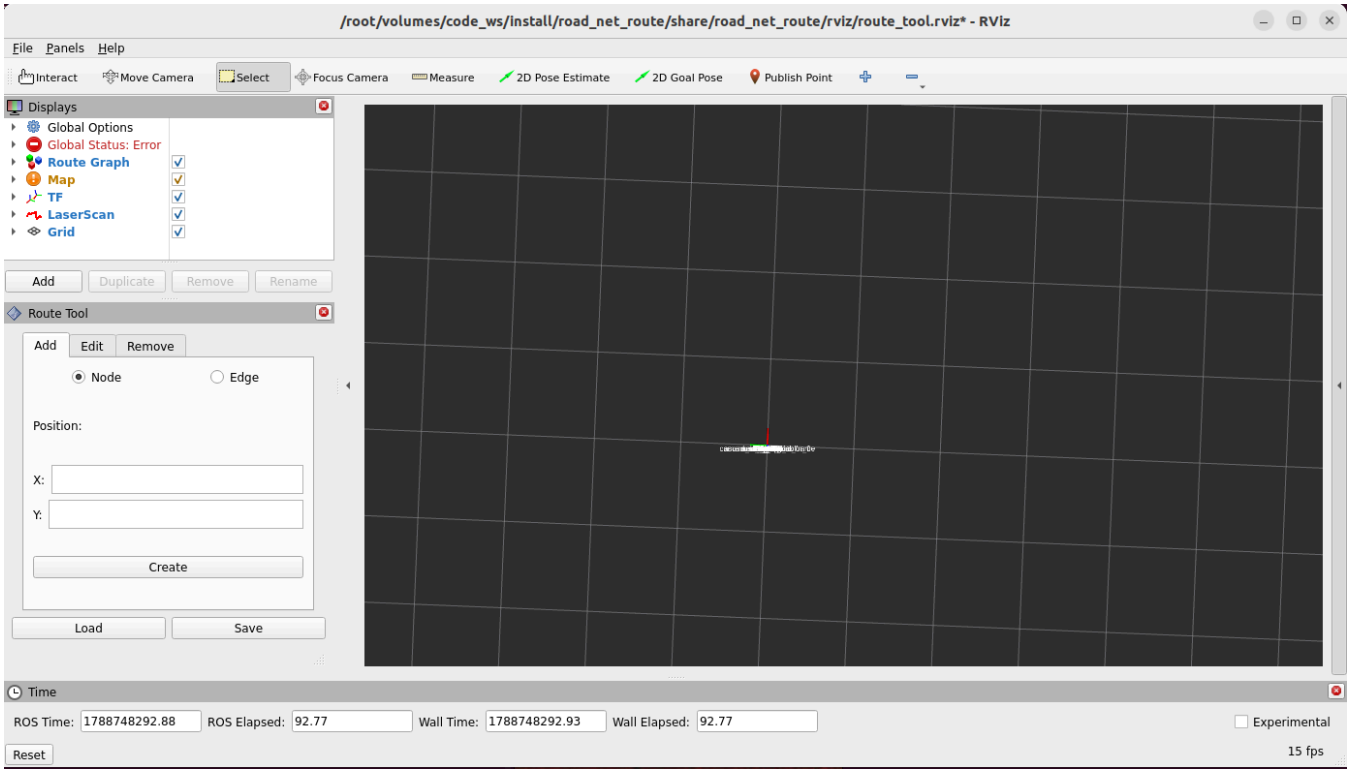
Terminal 3: Start Route Annotation RViz Tool Panel

Run annotation interface on board/virtual machine or container:

```
# Start annotation tool standalone panel
ros2 launch road_net_route route_tool.launch.py

# RDK and Raspberry Pi boards need to start RViz with integrated tools on virtual machine:
ros2 launch road_net_route rviz_only.launch.py tools:=true
```

Here demonstrating on Orin board,



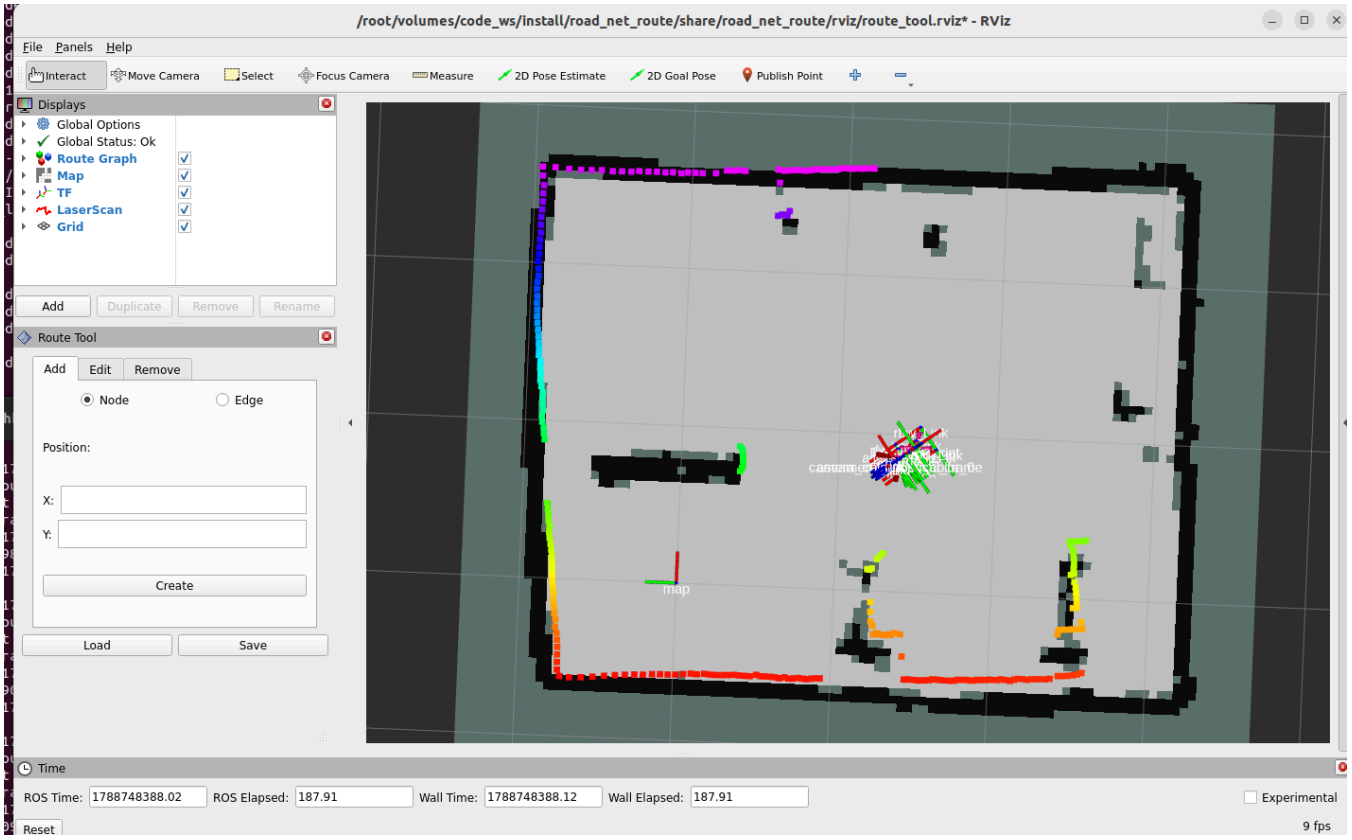
Terminal 4: Start Navigation Base Load Map (Cart Side)

```
# RDK X5 system
ros2 launch yahboomcar_nav navigation_teb_launch.py map:=/home/sunrise/map/map.yaml

# Or Jetson host
ros2 launch yahboomcar_nav navigation_teb_launch.py map:=/home/jetson/map/map.yaml

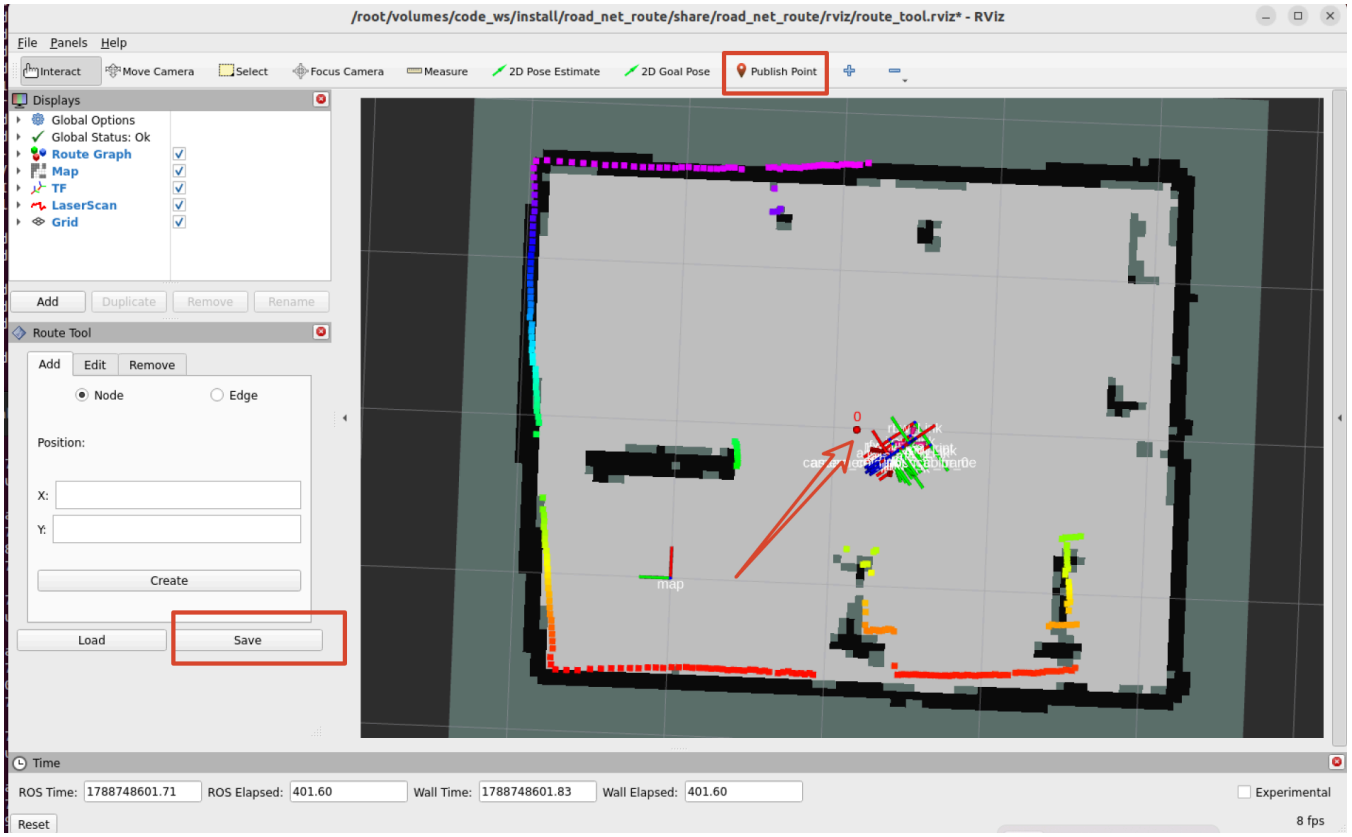
# Or pi5 host
ros2 launch yahboomcar_nav navigation_teb_launch.py map:=/root/map/map.yaml
```

In RViz, use the **2D Pose Estimate** button in the toolbar to give the cart an initial pose on the map, aligning the LiDAR scan outline precisely with the map black and white obstacle edges:



3.2 Interactively Add Waypoints (Nodes)

1. Click the `Publish Point` tool in RViz top toolbar;
2. Move mouse to the cart's current center position (or channel center you want to set as a station on the map), left-click;
3. The `Route Tool` panel at bottom-left will automatically read and fill in the X/Y coordinates of that point, click `Create` to create that waypoint:



4. Use keyboard control to move the cart to the next position that needs waypoint marking, repeat clicking `Publish Point` -> `Create`:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
jetson@yahboom:~/map$ ros2 run yahboomcar_ctrl yahboom_keyboard

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
t/T : x and y speed switch
s/S : stop keyboard control
space key, k : force stop

Arm Control (publish to 'arm_joint'):
1 - 6 : Select Joint 1~6
a / d : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
+ / - : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
0     : Reset all joints to initial pose

Angle Limits:
Joint 1~4 : 0° ~ 180°
Joint 5   : 0° ~ 270°
Joint 6   : 0° ~ 180°

anything else : stop smoothly
CTRL-C to quit

currently:      speed 0.2       turn 1.0
Current Joint Selection: Joint 1 (90°)
```

> ⚠ **Warning**
>
> If the laser outline slightly misaligns with the map due to wheel slipping during waypoint marking, you can manually use 2D Pose Estimate in RViz. After positioning converges and matches, calibrate the next waypoint.

5. Calibrate all target key stations in sequence (such as waypoints 0, 1, 2, ...):



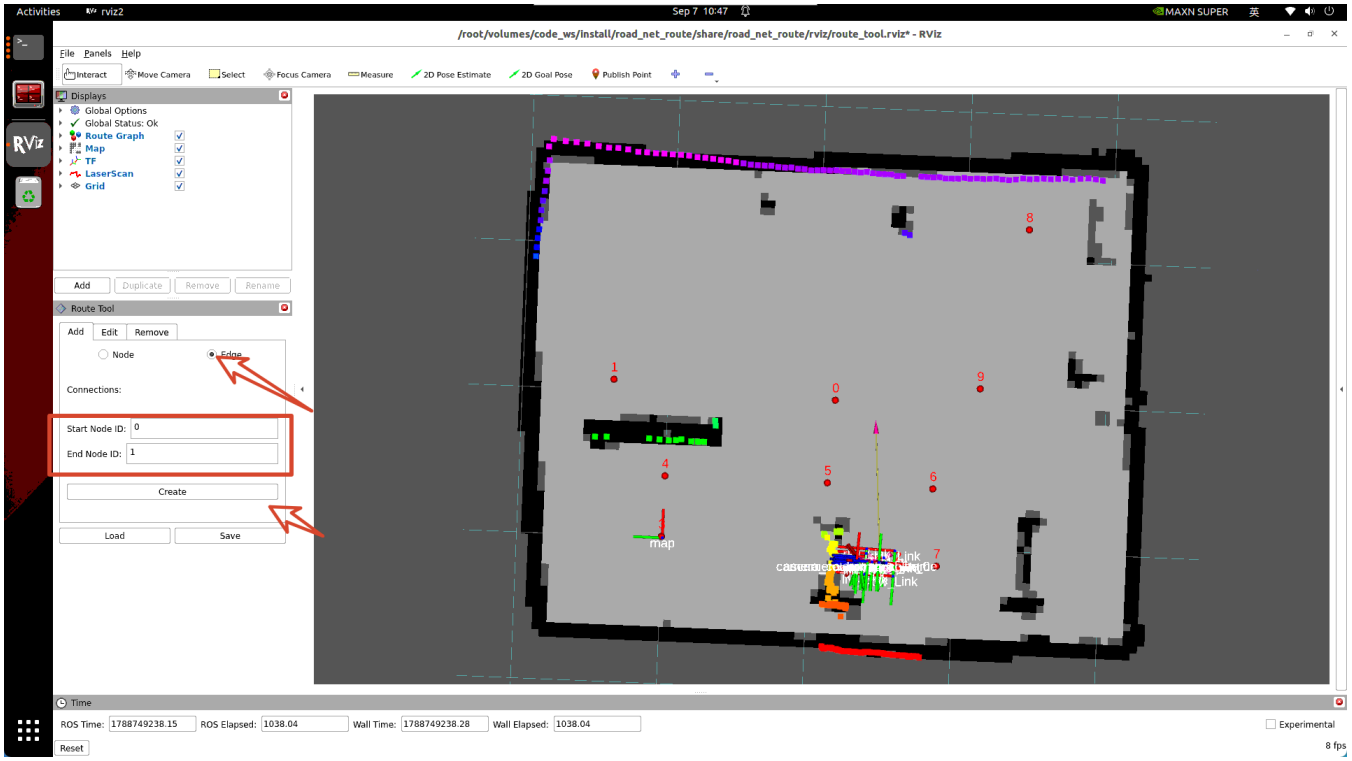
3.3 Interactively Add Directed Edges (Edges)

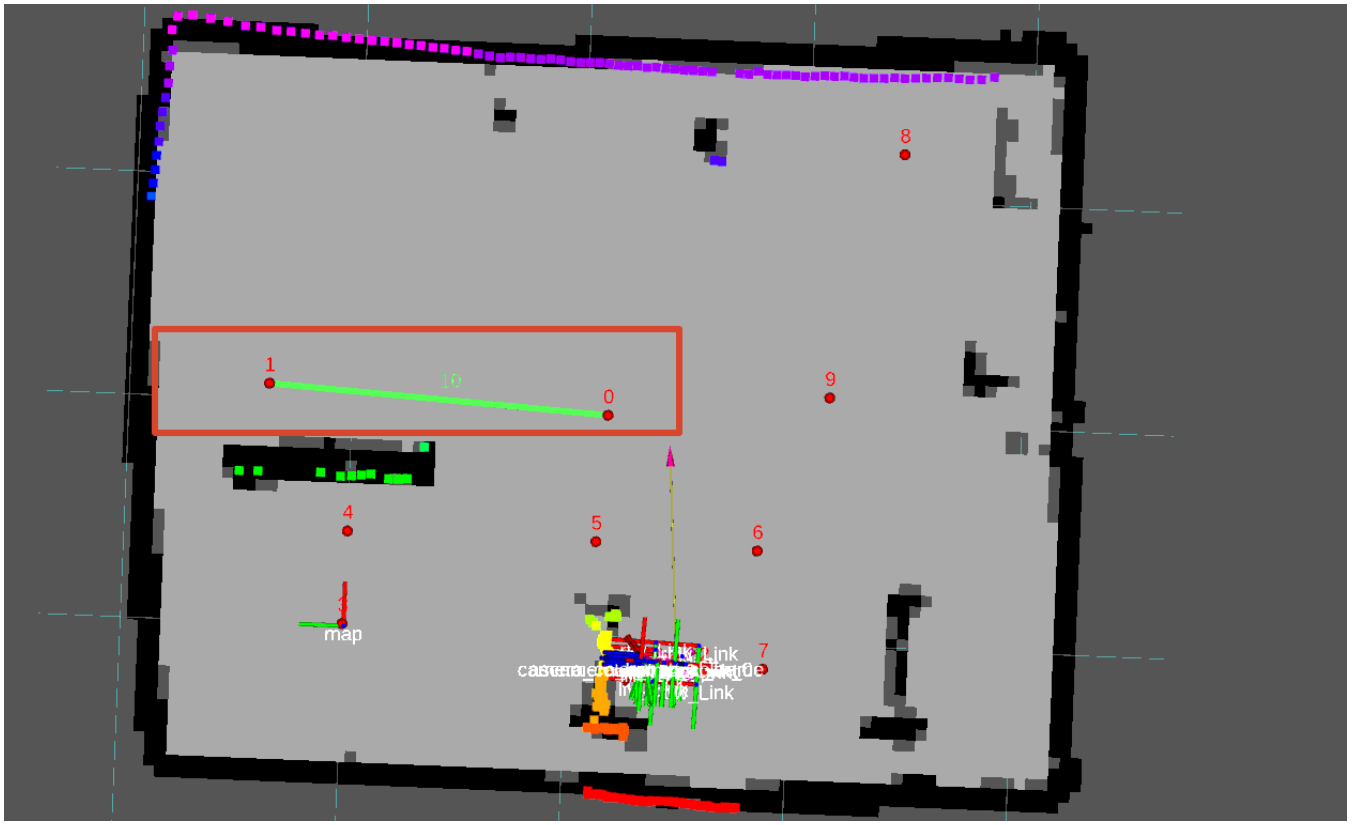
Route network is a typical **directed topology graph** structure. Only when points are connected by "edges" can the cart travel between them.

1. Switch to `Edge` page at the top of `Route Tool` tab;



2. Fill in the start node number in `Start Node ID` (such as 1), fill in the end node number in `End Node ID` (such as 2), click `Create`:

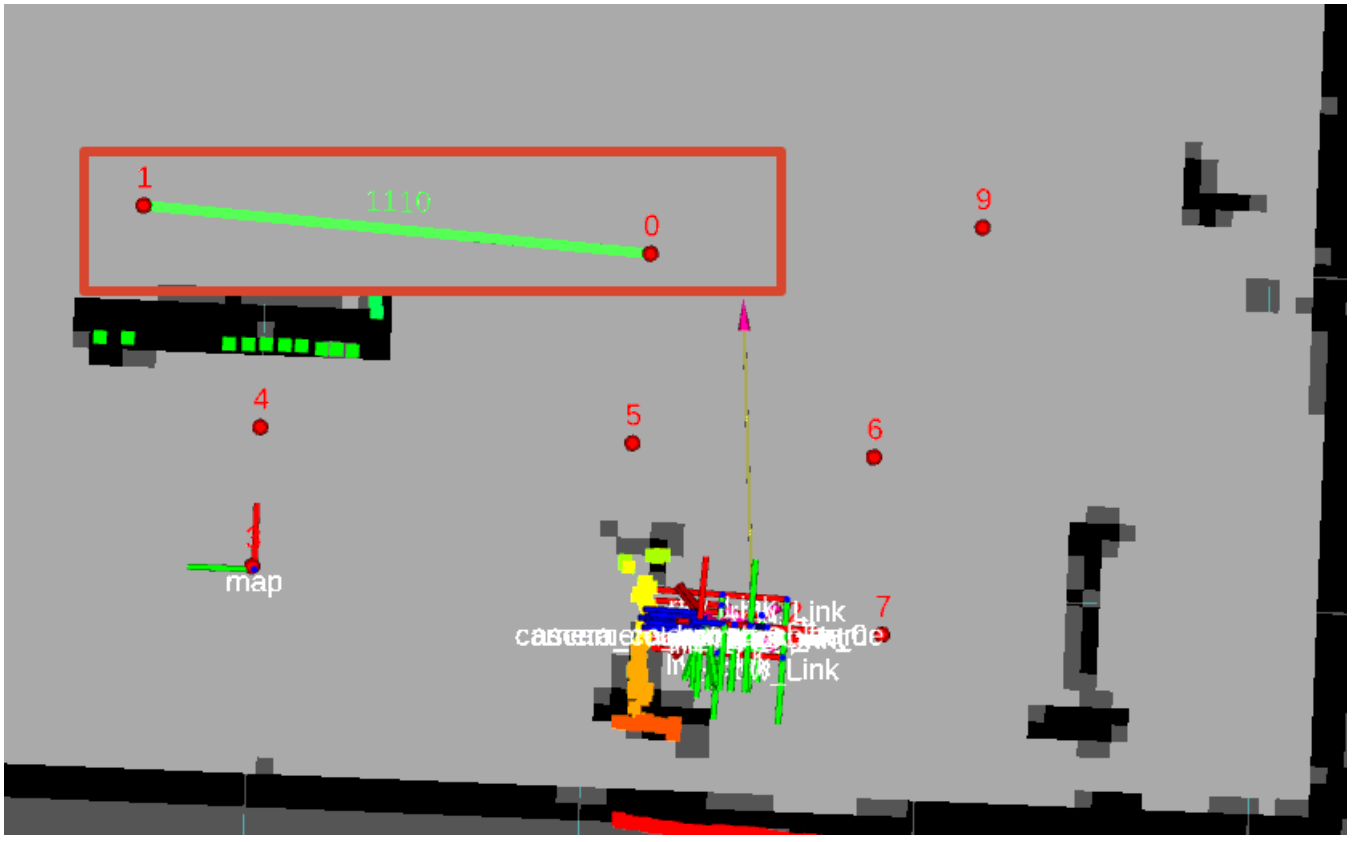




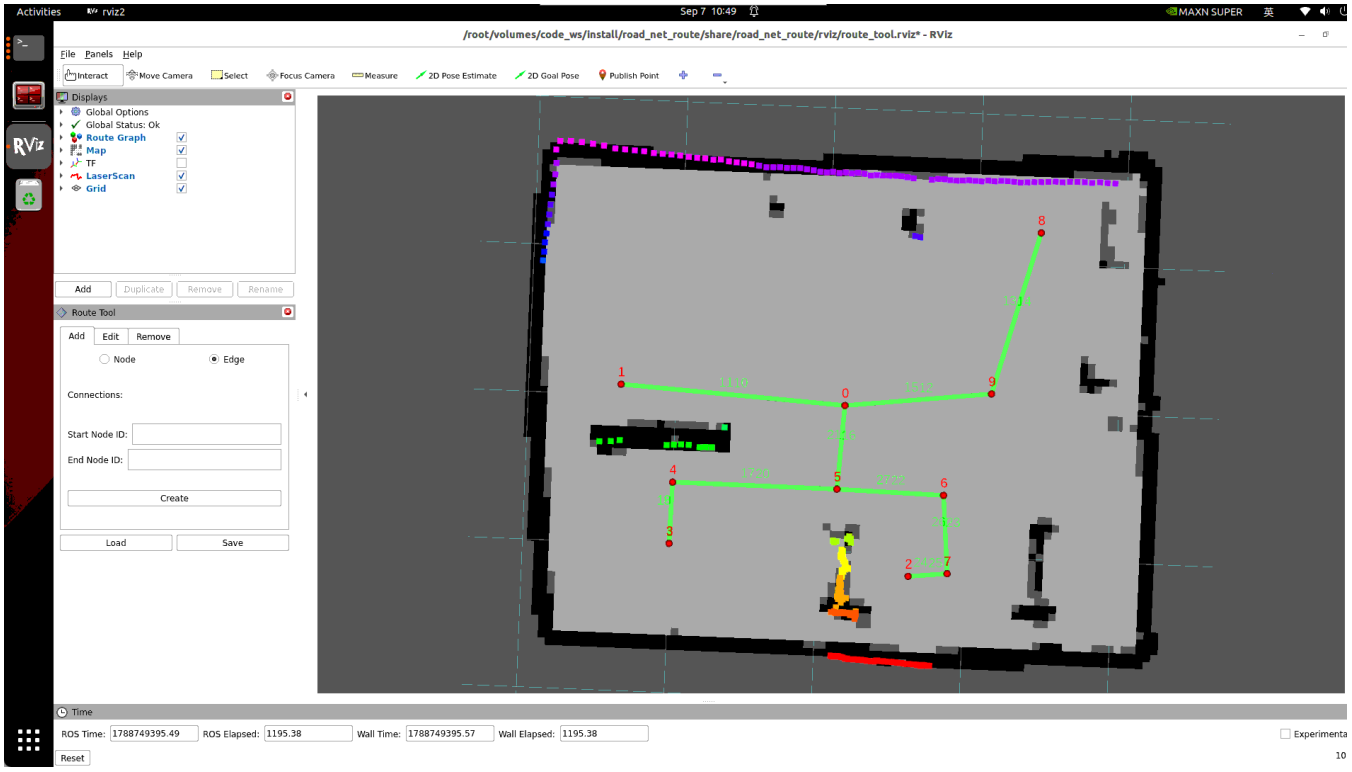
> **Important**
>
> **Special Note: Edges have direction!**
> Creating 1 -> 2 means this is a **one-way road** (only allowed from 1 to 2, no reverse). If this road allows two-way traffic, you must create another edge 2 -> 1!

3. Establish reverse edge for two-way traffic lanes:



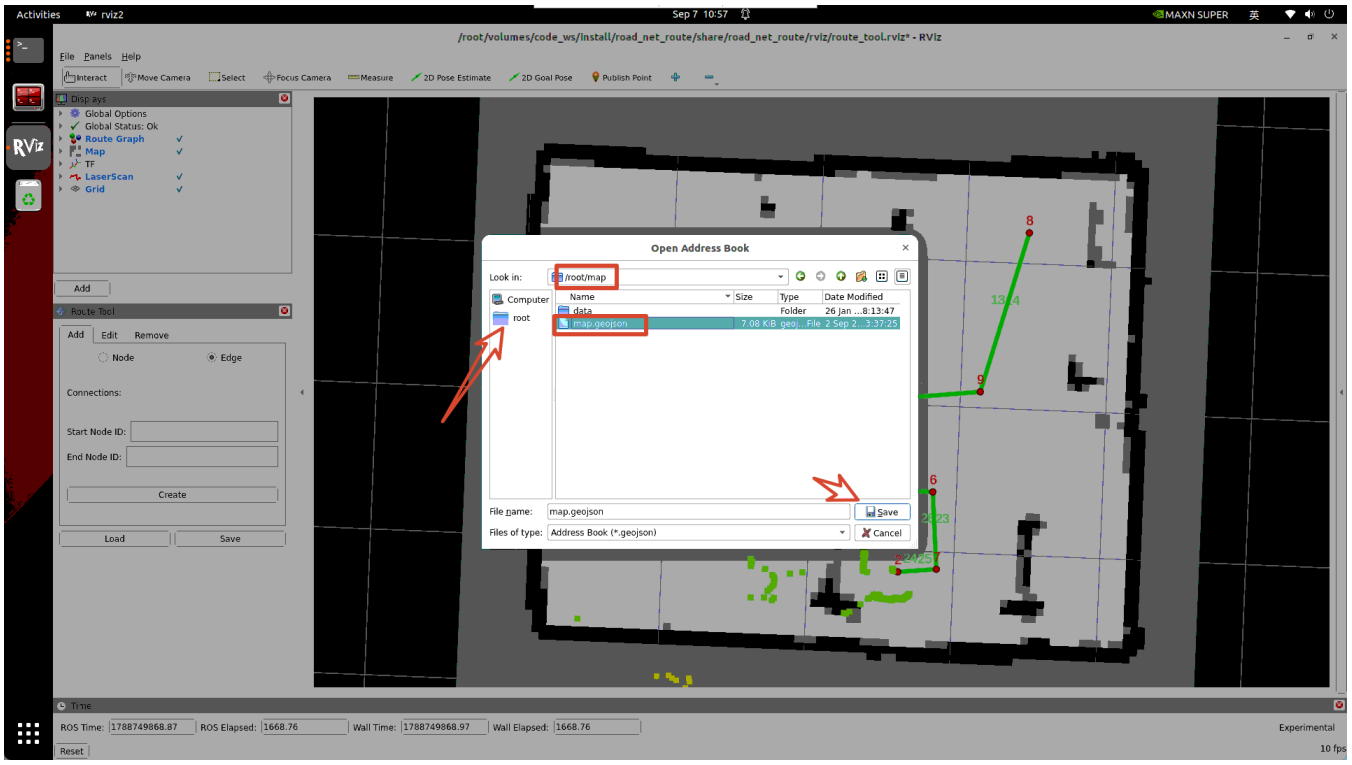


4. Complete connections for all passable road segments in the venue in sequence:

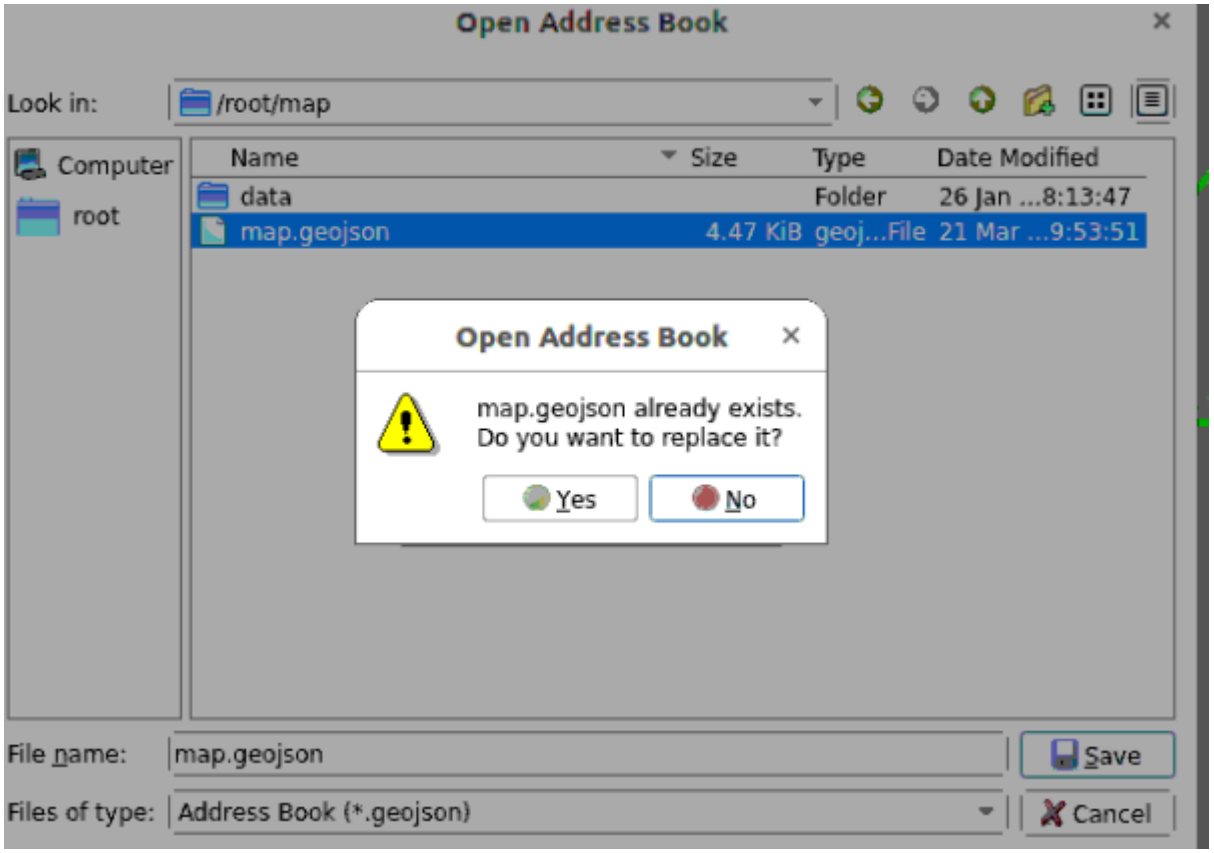


3.4 Save Route Network Description File (`.geojson`)

1. Click `Save` button in `Route Tool` panel;
2. In the pop-up file save dialog, name the file `map.geojson` (recommended to save in `~/map/` directory with same name as grid map): may lag and wait a while



Select overwrite



3. Terminal prints successful JSON save record:

```
rviz2-1] [INFO] [1788355922.422582445] [route_tool_node]: Adding edge from 5 to

rviz2-1] [INFO] [1788355925.911016578] [route_tool_node]: Adding edge from 4 to

rviz2-1] [INFO] [1788355930.540818916] [route_tool_node]: Adding edge from 3 to

rviz2-1] [INFO] [1788356245.903659911] [route_tool_node]: Save graph to: /root/m
p/map.geojson
rviz2-1] [INFO] [1788356245.903863341] [route_tool_node]: Saving graph file from
/root/map/map.geojson, by parser nav2_route::GeoJsonGraphFileSaver
```